

---

## SENTIMENT ANALYSIS SERVICE

### Deep Engineering Review and Ownership Handbook

CTO review, due diligence, production readiness, handover, onboarding

|                           |                    |
|---------------------------|--------------------|
| Tech Lead                 | Le Quang Tuyen     |
| ML Engineering Lead       | Duong Binh An      |
| AI Engineering Lead       | Do Quoc Trung      |
| Solution Engineering Lead | Duong Hong Quan    |
| Backend Engineering Lead  | Pham Duc Long      |
| UI/UX Engineering Lead    | Nguyen Le Hong Nhi |

#### Repository truth order used in this handbook:

1. Source code
  2. Runtime configuration
  3. Infrastructure manifests
  4. Tests
  5. Existing reports
  6. README and legacy docs
- 
-

## Contents

---

|          |                                                |           |
|----------|------------------------------------------------|-----------|
| <b>1</b> | <b>Part 1 - Executive Assessment</b>           | <b>3</b>  |
| 1.1      | Business Problem . . . . .                     | 3         |
| 1.2      | Product Purpose . . . . .                      | 3         |
| 1.3      | Technical Objective . . . . .                  | 3         |
| 1.4      | End Users . . . . .                            | 3         |
| 1.5      | Value Proposition . . . . .                    | 3         |
| 1.6      | Maturity Classification . . . . .              | 4         |
| 1.7      | Implementation-based Justification . . . . .   | 4         |
| <b>2</b> | <b>Part 2 - System Mental Model</b>            | <b>4</b>  |
| 2.1      | Business Mental Model . . . . .                | 4         |
| 2.2      | Product Mental Model . . . . .                 | 4         |
| 2.3      | Engineering Mental Model . . . . .             | 5         |
| 2.4      | Operations Mental Model . . . . .              | 5         |
| 2.5      | AI/ML Mental Model . . . . .                   | 5         |
| 2.6      | Plain-language Explanation . . . . .           | 6         |
| <b>3</b> | <b>Part 3 - Complete End-to-End Flow</b>       | <b>6</b>  |
| 3.1      | Flow Table . . . . .                           | 6         |
| 3.2      | Primary Runtime Sequence . . . . .             | 7         |
| 3.3      | Training-to-Serving Sequence . . . . .         | 7         |
| <b>4</b> | <b>Part 4 - Repository Reverse Engineering</b> | <b>7</b>  |
| 4.1      | Major Folder Analysis . . . . .                | 7         |
| 4.2      | Folder Dependency Graph . . . . .              | 8         |
| 4.3      | Runtime Dependency Graph . . . . .             | 8         |
| 4.4      | Training Dependency Graph . . . . .            | 9         |
| 4.5      | Deployment Dependency Graph . . . . .          | 9         |
| <b>5</b> | <b>Part 5 - File-by-File Critical Analysis</b> | <b>9</b>  |
| 5.1      | Top 50 Most Important Files . . . . .          | 9         |
| 5.2      | Criticality Notes . . . . .                    | 12        |
| <b>6</b> | <b>Part 6 - Execution Trace</b>                | <b>12</b> |
| 6.1      | Application Startup . . . . .                  | 12        |
| 6.2      | Health Check . . . . .                         | 13        |
| 6.3      | Predict . . . . .                              | 13        |
| 6.4      | Explain . . . . .                              | 13        |
| 6.5      | Batch Predict . . . . .                        | 13        |
| 6.6      | Evaluation . . . . .                           | 14        |
| 6.7      | Training . . . . .                             | 14        |
| 6.8      | ONNX Export . . . . .                          | 14        |

|           |                                                |           |
|-----------|------------------------------------------------|-----------|
| 6.9       | Sequence Diagrams . . . . .                    | 15        |
| <b>7</b>  | <b>Part 7 - Frontend Handbook</b>              | <b>15</b> |
| 7.1       | Owner . . . . .                                | 15        |
| 7.2       | Architecture . . . . .                         | 16        |
| 7.3       | Components . . . . .                           | 16        |
| 7.4       | Services and State Flow . . . . .              | 16        |
| 7.5       | Explainability UI . . . . .                    | 16        |
| 7.6       | Batch Upload UI . . . . .                      | 16        |
| 7.7       | Key Risks . . . . .                            | 16        |
| 7.8       | Extension Points . . . . .                     | 16        |
| <b>8</b>  | <b>Part 8 - Backend Handbook</b>               | <b>17</b> |
| 8.1       | Owner . . . . .                                | 17        |
| 8.2       | FastAPI Structure . . . . .                    | 17        |
| 8.3       | Contracts and Validation . . . . .             | 17        |
| 8.4       | API Ownership Map . . . . .                    | 17        |
| 8.5       | Error Handling . . . . .                       | 17        |
| 8.6       | Backend Gaps . . . . .                         | 18        |
| <b>9</b>  | <b>Part 9 - AI Inference Handbook</b>          | <b>18</b> |
| 9.1       | Owner . . . . .                                | 18        |
| 9.2       | Pipeline . . . . .                             | 18        |
| 9.3       | Tokenization and Tensors . . . . .             | 18        |
| 9.4       | ONNX Runtime . . . . .                         | 18        |
| 9.5       | Prediction Logic . . . . .                     | 18        |
| 9.6       | ABSA . . . . .                                 | 19        |
| 9.7       | SHAP . . . . .                                 | 19        |
| 9.8       | Language Detection . . . . .                   | 19        |
| 9.9       | Latency Analysis . . . . .                     | 19        |
| <b>10</b> | <b>Part 10 - ML Handbook</b>                   | <b>19</b> |
| 10.1      | Owner . . . . .                                | 19        |
| 10.2      | Dataset Strategy . . . . .                     | 19        |
| 10.3      | DVC . . . . .                                  | 19        |
| 10.4      | MLflow . . . . .                               | 20        |
| 10.5      | LoRA and PEFT . . . . .                        | 20        |
| 10.6      | Evaluation and Fairness . . . . .              | 20        |
| 10.7      | Training Lifecycle . . . . .                   | 20        |
| 10.8      | ML Gaps . . . . .                              | 20        |
| <b>11</b> | <b>Part 11 - Solution Engineering Handbook</b> | <b>21</b> |
| 11.1      | Owner . . . . .                                | 21        |
| 11.2      | End-to-End Integration . . . . .               | 21        |

|           |                                                                         |           |
|-----------|-------------------------------------------------------------------------|-----------|
| 11.3      | Deployment Architecture . . . . .                                       | 21        |
| 11.4      | Monitoring Strategy . . . . .                                           | 21        |
| 11.5      | Operational Readiness Assessment . . . . .                              | 21        |
| <b>12</b> | <b>Part 12 - Ownership Matrix</b>                                       | <b>21</b> |
| 12.1      | Folder Ownership Matrix . . . . .                                       | 22        |
| 12.2      | Service Ownership Matrix . . . . .                                      | 22        |
| 12.3      | Operational, Security, Documentation, and Knowledge Ownership . . . . . | 22        |
| <b>13</b> | <b>Part 13 - RACI Matrix</b>                                            | <b>22</b> |
| <b>14</b> | <b>Part 14 - Data and Model Lineage</b>                                 | <b>23</b> |
| <b>15</b> | <b>Part 15 - Security Review</b>                                        | <b>23</b> |
| <b>16</b> | <b>Part 16 - Performance Review</b>                                     | <b>24</b> |
| 16.1      | Startup Time . . . . .                                                  | 24        |
| 16.2      | Memory Usage . . . . .                                                  | 24        |
| 16.3      | CPU Usage . . . . .                                                     | 24        |
| 16.4      | Inference Latency . . . . .                                             | 24        |
| 16.5      | Batch Throughput . . . . .                                              | 24        |
| 16.6      | SHAP Cost . . . . .                                                     | 25        |
| 16.7      | ABSA Cost . . . . .                                                     | 25        |
| 16.8      | Scalability Limits . . . . .                                            | 25        |
| <b>17</b> | <b>Part 17 - Operations Handbook</b>                                    | <b>25</b> |
| <b>18</b> | <b>Part 18 - Technical Debt Register</b>                                | <b>26</b> |
| <b>19</b> | <b>Part 19 - Organizational Review</b>                                  | <b>26</b> |
| 19.1      | Team Structure . . . . .                                                | 26        |
| 19.2      | Knowledge Distribution and Bus Factor . . . . .                         | 26        |
| 19.3      | Ownership Gaps . . . . .                                                | 27        |
| 19.4      | Recommendations . . . . .                                               | 27        |
| <b>20</b> | <b>Part 20 - Production Readiness Scorecard</b>                         | <b>27</b> |
| <b>21</b> | <b>Part 21 - Roadmap</b>                                                | <b>27</b> |
| 21.1      | 3-Month Roadmap . . . . .                                               | 27        |
| 21.2      | 6-Month Roadmap . . . . .                                               | 28        |
| 21.3      | 12-Month Roadmap . . . . .                                              | 28        |
| <b>22</b> | <b>Part 22 - Onboarding Handbook</b>                                    | <b>28</b> |
| <b>23</b> | <b>Part 23 - CTO Review</b>                                             | <b>28</b> |
| 23.1      | Would I approve production launch? . . . . .                            | 29        |

|                                                      |           |
|------------------------------------------------------|-----------|
| 23.2 Would I approve enterprise customers? . . . . . | 29        |
| 23.3 Would I approve scaling? . . . . .              | 29        |
| 23.4 What concerns me most? . . . . .                | 29        |
| 23.5 What would be my first investment? . . . . .    | 29        |
| <b>Implementation Discrepancies</b>                  | <b>29</b> |

## 1 Part 1 - Executive Assessment

---

### 1.1 Business Problem

The business problem is converting free-form customer feedback into operationally useful signals fast enough to support product analytics, customer-support triage, service-quality monitoring, and AI demonstration use cases. Manual reading does not scale and does not provide structured outputs.

### 1.2 Product Purpose

The repository delivers:

- an end-user chat and batch UI for sentiment analysis
- a FastAPI inference service for prediction, explanation, batch scoring, health, metrics, and evaluation trigger flows
- an offline ML pipeline for data download, preprocessing, validation, finetuning, evaluation, and ONNX export

### 1.3 Technical Objective

The technical objective is to prove a complete ML-in-production chain:

- DVC for data-pipeline stage orchestration
- MLflow for experiment and evaluation tracking
- PEFT/LoRA for finetuned adapters
- ONNX Runtime for optimized serving
- Prometheus and Grafana for observability
- Docker Compose for integrated deployment

### 1.4 End Users

- product analysts and business stakeholders reviewing sentiment
- demo users interacting with chat-style sentiment explanation
- engineers operating the backend and training flows
- future owners who must maintain runtime and training artifacts

### 1.5 Value Proposition

The value is not only prediction accuracy. The stronger proposition is that the system packages inference, batch handling, explainability, monitoring, and artifact lifecycle into one coherent delivery surface.

## 1.6 Maturity Classification

| Stage            | Assessment                                                                                                          |
|------------------|---------------------------------------------------------------------------------------------------------------------|
| Prototype        | Exceeded. The repo includes contracts, tests, monitoring, and deployment manifests.                                 |
| MVP              | Exceeded. End-user flows exist for single prediction, explanation, and batch upload.                                |
| Advanced MVP     | <b>Current state.</b> The codebase supports offline training, online inference, ONNX export, observability, and CI. |
| Pre-Production   | Partial. Security controls, durable jobs, and operational hardening are incomplete.                                 |
| Production Ready | Not approved. Security, reliability, and ownership controls are below enterprise standard.                          |

## 1.7 Implementation-based Justification

- `src/main.py` exposes working inference and evaluation endpoints.
- `src/model/baseline.py` implements real model orchestration, ABSA, SHAP, and ONNX paths.
- `dvc.yaml` encodes a real multi-stage ML pipeline.
- `docker-compose.yml` wires runtime, frontend, Prometheus, Grafana, MLflow, and training profile.
- `/batch_status/{job_id}` is mocked, so durable async batch processing is **Not Implemented**.
- authentication and authorization are **Not Implemented**.

## 2 Part 2 - System Mental Model

### 2.1 Business Mental Model

```

1 flowchart LR
2   U[User or Analyst]
3   FE[Frontend Experience]
4   API[Inference Service]
5   INS[Structured Insight]
6   OPS[Monitoring and Review]
7   TRAIN[Retraining and Export]
8   U --> FE --> API --> INS --> OPS --> TRAIN

```

### 2.2 Product Mental Model

```

1 flowchart TB
2   UI[Angular App]
3   P1[Single Prediction]
4   P2[Explain SHAP]
5   P3[Batch CSV]
6   P4[Status and Health]

```

```
7 P5[Operational Metrics]
8 UI --> P1
9 UI --> P2
10 UI --> P3
11 UI --> P4
12 P1 --> API[FastAPI]
13 P2 --> API
14 P3 --> API
15 P4 --> API
16 API --> P5
```

## 2.3 Engineering Mental Model

```
1 flowchart LR
2 FE[app/sentiment-analysis-chatbot]
3 GW[Nginx /api proxy]
4 BE[src/main.py]
5 INF[src/model]
6 DATA[src/data]
7 TRAIN[src/scripts + src/training]
8 OBS[src/monitoring + infra]
9 FE --> GW --> BE --> INF
10 DATA --> TRAIN --> INF
11 BE --> OBS
```

## 2.4 Operations Mental Model

```
1 flowchart LR
2 Browser --> Frontend[:80]
3 Frontend --> Nginx[nginx.conf]
4 Nginx --> FastAPI[:8000]
5 FastAPI --> Metrics[/metrics]
6 Metrics --> Prometheus[:9091]
7 Prometheus --> Grafana[:3000]
8 FastAPI --> Eval[/evaluate]
9 Eval --> MLflow[:5005]
```

## 2.5 AI/ML Mental Model

```
1 flowchart LR
2 RAW[data/raw]
3 PREP[src/data/pipeline.py]
4 VALID[src/data/validators.py]
5 FT[src/scripts/run_finetuning.py]
6 EVAL[src/model/evaluate.py and evaluate_finetuned.py]
7 EXPORT[src/scripts/export_onnx.py]
8 SERVE[src/model/baseline.py]
9 RAW --> PREP --> VALID --> FT --> EVAL --> EXPORT --> SERVE
```



## 2.6 Plain-language Explanation

For business stakeholders, the system reads text and returns sentiment plus supporting detail. For engineers, the system is a frontend plus a backend plus an embedded model runtime plus an offline training toolchain. For new team members, the easiest mental model is this: `app/` is the user surface, `src/main.py` is the runtime entrypoint, `src/model/` is the inference core, `src/data/` and `src/training/` are the artifact factory, and `infra/` plus `docker-compose.yml` are the operating shell around all of it.

## 3 Part 3 - Complete End-to-End Flow

### 3.1 Flow Table

| Step            | Objective                                 | Implementation                                                                          | Owner                   | Dependencies                             | Failure Modes                               |
|-----------------|-------------------------------------------|-----------------------------------------------------------------------------------------|-------------------------|------------------------------------------|---------------------------------------------|
| User            | submit text or CSV                        | chat input, voice transcription, or file upload in Angular                              | UI/UX                   | browser, local state                     | invalid input, upload abandonment           |
| Frontend        | collect and send requests                 | <code>app.component.ts</code> , <code>components</code> , <code>message.model.ts</code> | UI/UX                   | Angular, HttpClient, Nginx               | stale UI state, contract drift              |
| API Gateway     | route UI traffic                          | Nginx <code>/api/</code> proxy in <code>nginx.conf</code>                               | Solution                | frontend container, backend service DNS  | timeout, buffering issues, file-size issues |
| Backend         | validate and route                        | <code>src/main.py</code> , Pydantic schemas                                             | Backend                 | FastAPI, contracts, model dependency     | 400s, 500s, startup failure                 |
| Inference Layer | normalize language and execute model path | language resolution plus <code>BaselineModelInference</code>                            | AI                      | model artifacts, tokenizer, runtime mode | unsupported language, load failure          |
| Model           | produce logits, labels, ABSA, SHAP        | HF or ONNX plus zero-shot classifier and SHAP explainer                                 | AI                      | ONNX/HF artifacts, CPU/GPU, libraries    | latency spikes, artifact mismatch           |
| Response        | return structured payload                 | response models from <code>contracts/schemas.py</code>                                  | Backend                 | model outputs                            | schema mismatch, missing fields             |
| Monitoring      | observe traffic and latency               | middleware and Prometheus / Grafana stack                                               | Solution                | metrics endpoint, scrape config          | silent monitoring blind spot                |
| Feedback        | inspect reports and dashboard signals     | Grafana panels, MLflow, JSON reports                                                    | Solution + ML           | + observability stack, report files      | stale metrics, missing artifact context     |
| Training        | improve runtime artifacts                 | DVC stages and training scripts                                                         | ML                      | raw data, params, libraries              | data failure, imbalance issues              |
| Deployment      | move serving artifact into runtime image  | Dockerfiles, Compose, DVC pull, ONNX export                                             | Solution + AI + Backend | image build, secrets, model files        | bad image, bad artifact, secret leak        |

### 3.2 Primary Runtime Sequence

```

1 sequenceDiagram
2   participant User
3   participant UI as Angular UI
4   participant GW as Nginx Gateway
5   participant API as FastAPI
6   participant LD as LanguageDetector
7   participant INF as BaselineModelInference
8   participant OBS as Prometheus Metrics
9   User->>UI: submit text
10  UI->>GW: POST /api/predict
11  GW->>API: forward request
12  API->>LD: resolve_request_language()
13  API->>INF: predict_single(text, lang)
14  INF-->>API: PredictionResult
15  API->>OBS: increment counters and latency
16  API-->>GW: PredictResponse
17  GW-->>UI: JSON response
18  UI-->>User: sentiment and aspects

```

### 3.3 Training-to-Serving Sequence

```

1 sequenceDiagram
2   participant DVC
3   participant Data as src/data
4   participant Train as run_finetuning.py
5   participant Eval as evaluate_finetuned.py
6   participant Export as export_onnx.py
7   participant Runtime as Docker runtime image
8   DVC->>Data: download and preprocess
9   Data->>Train: produce normalized train data
10  Train->>Eval: adapters for validation
11  Eval->>Export: approved finetuned adapters
12  Export->>Runtime: ONNX artifacts

```

## 4 Part 4 - Repository Reverse Engineering

### 4.1 Major Folder Analysis

| Folder     | Purpose                                       | Runtime<br>vance                          | Rele-<br>vance | Business<br>vance | Rele-<br>vance | Owner        | Risk   |
|------------|-----------------------------------------------|-------------------------------------------|----------------|-------------------|----------------|--------------|--------|
| app/       | frontend user experience and API client       | direct                                    |                | direct            |                | UI/UX Lead   | Medium |
| contracts/ | shared request, response, and model contracts | direct                                    |                | direct            |                | Backend Lead | High   |
| data/      | raw, processed, and eval data artifacts       | indirect at run-time, direct for training |                | high              |                | ML Lead      | High   |

| Folder          | Purpose                                              | Runtime<br>vance  | Rele-<br>vance | Business<br>vance | Rele-<br>vance | Owner         | Risk     |
|-----------------|------------------------------------------------------|-------------------|----------------|-------------------|----------------|---------------|----------|
| docs/           | reports, handbook out-puts, legacy architecture docs | indirect          |                | medium            |                | Tech Lead     | Low      |
| infra/          | Prometheus and Grafana provisioning                  | direct            |                | medium            |                | Solution Lead | Medium   |
| mlruns/         | local MLflow tracking store                          | indirect          |                | medium            |                | ML Lead       | Low      |
| models/         | adapters and ONNX artifacts                          | direct            |                | high              |                | AI Lead       | High     |
| notebooks/      | ad hoc or Colab execution path                       | indirect          |                | medium            |                | ML Lead       | Medium   |
| reports/        | generated evaluation and benchmark reports           | indirect          |                | medium            |                | ML Lead       | Low      |
| src/data/       | download, preprocess, validate logic                 | training-critical |                | high              |                | ML Lead       | High     |
| src/model/      | runtime inference and export logic                   | runtime-critical  |                | high              |                | AI Lead       | Critical |
| src/monitoring/ | API metrics middleware                               | runtime-critical  |                | medium            |                | Solution Lead | Medium   |
| src/scripts/    | CLI entrypoints for training and export              | training-critical |                | high              |                | ML Lead       | High     |
| src/training/   | reusable training internals                          | training-critical |                | high              |                | ML Lead       | High     |
| tests/          | regression safety net                                | indirect          |                | high              |                | Tech Lead     | Medium   |

## 4.2 Folder Dependency Graph

```

1 flowchart LR
2   app --> contracts
3   app --> src_main[src/main.py]
4   src_main --> contracts
5   src_main --> src_model[src/model]
6   src_model --> models
7   src_scripts[src/scripts] --> src_training[src/training]
8   src_scripts --> src_data[src/data]
9   src_scripts --> src_model
10  infra --> src_main
11  docker[docker-compose.yml] --> app
12  docker --> src_main
13  docker --> infra

```

## 4.3 Runtime Dependency Graph

```

1 flowchart LR
2   Browser --> Angular
3   Angular --> Nginx

```

```

4   Nginx --> FastAPI
5   FastAPI --> Contracts
6   FastAPI --> Monitoring
7   FastAPI --> BaselineModel
8   BaselineModel --> ONNXArtifacts
9   BaselineModel --> ABSAZeroShot
10  BaselineModel --> SHAP

```

#### 4.4 Training Dependency Graph

```

1  flowchart LR
2    RawData --> Downloader
3    Downloader --> Preprocess
4    Preprocess --> Validator
5    Validator --> Finetuning
6    Finetuning --> Evaluation
7    Evaluation --> MLflow
8    Finetuning --> Adapters
9    Adapters --> ONNXExport

```

#### 4.5 Deployment Dependency Graph

```

1  flowchart LR
2    Dockerfile --> RuntimeImage
3    Dockerfile_train[Dockerfile.train] --> TrainingImage
4    dvc_pull[DVC pull in model stage] --> RuntimeImage
5    docker_compose --> RuntimeImage
6    docker_compose --> FrontendImage
7    docker_compose --> Prometheus
8    docker_compose --> Grafana
9    docker_compose --> MLflow

```

## 5 Part 5 - File-by-File Critical Analysis

### 5.1 Top 50 Most Important Files

| # | File                        | Why it exists                      | When it executes                       | What breaks if removed           | Owner   | Risk     |
|---|-----------------------------|------------------------------------|----------------------------------------|----------------------------------|---------|----------|
| 1 | src/main.py                 | runtime entrypoint and API surface | startup and every request              | service does not run             | Backend | Critical |
| 2 | src/model/baseline.py       | core inference orchestration       | startup, predict, explain, batch, eval | no prediction path               | AI      | Critical |
| 3 | src/model/config.py         | model mode and artifact paths      | startup and model creation             | runtime cannot resolve artifacts | AI      | High     |
| 4 | src/model/onnx_inference.py | ONNX session wrapper               | onnx startup and inference             | ONNX mode fails                  | AI      | High     |

| #  | File                                                      | Why it exists                           | When it executes                          | What breaks if removed                  | Owner | Risk     |
|----|-----------------------------------------------------------|-----------------------------------------|-------------------------------------------|-----------------------------------------|-------|----------|
| 5  | src/model/onnx_exporter_and_adapters                      | merge and export adapters               | export workflow                           | no ONNX promotion path                  | AI    | High     |
| 6  | src/model/evaluate_baseline_evaluation_and_MLflow_logging | baseline evaluation and MLflow logging  | offline eval and background eval          | no baseline scoring path                | ML    | High     |
| 7  | src/model/language_retrieval_language_fallback            | retrieve language fallback              | predict and explain without explicit lang | unsupported auto-detect path            | AI    | Medium   |
| 8  | src/model/device.py                                       | device selection                        | model load and export                     | runtime hardware selection breaks       | AI    | Medium   |
| 9  | src/model/download_pretrained_HF_assets                   | preload HF assets in image build        | Docker build                              | image startup becomes network-dependent | AI    | Medium   |
| 10 | src/data/pipeline.py                                      | preprocessing chain                     | DVC preprocess stage                      | processed data not generated            | ML    | High     |
| 11 | src/data/downloader.py                                    | raw dataset acquisition                 | download stages                           | training inputs unavailable             | ML    | High     |
| 12 | src/data/validator.py                                     | quality gates and MLflow data reporting | validate stage                            | data quality checks disappear           | ML    | High     |
| 13 | src/scripts/run_finetuning_Cli.py                         | finetuning CLI entrypoint               | training stage                            | no adapter training path                | ML    | Critical |
| 14 | src/scripts/evaluate_finetuned_adapter.py                 | finetuned adapter evaluation            | DVC evaluate stage                        | no post-finetune scorecard              | ML    | High     |
| 15 | src/scripts/export_ONNX.py                                | ONNX export CLI wrapper                 | export stage                              | no deployment artifact export command   | AI    | High     |
| 16 | src/scripts/prepare_eval_datasets                         | build eval datasets                     | prepare_eval stage                        | finetuned eval inputs missing           | ML    | Medium   |
| 17 | src/scripts/benchmark_benchmark.py                        | benchmark PyTorch vs ONNX modes         | benchmark stage                           | no benchmark refresh path               | AI    | Medium   |
| 18 | src/training/task_registry.py                             | task registry for sentiment and sarcasm | training and eval                         | task metadata fragments                 | ML    | High     |
| 19 | src/training/dataset_cleanup_and_data_sampler.py          | dedup and over-sampling helpers         | training                                  | split and balance behavior drifts       | ML    | High     |
| 20 | src/training/metrics_fairness-aware_metrics.py            | fairness-aware metrics payloads         | evaluation                                | no consistent per-language metrics      | ML    | High     |
| 21 | src/training/weighted_loss.py                             | weighted loss trainer                   | training when class weights enabled       | imbalance mitigation weakens            | ML    | Medium   |
| 22 | src/training/class_inverses.py                            | inverse frequency class weights         | training                                  | weighted loss inputs break              | ML    | Medium   |

| #  | File                                             | Why it exists                                  | When it executes                        | What breaks if removed                  | Owner         | Risk     |
|----|--------------------------------------------------|------------------------------------------------|-----------------------------------------|-----------------------------------------|---------------|----------|
| 23 | src/training/lora_config.py                      | LoRA config creation                           | training                                | adapter config cannot be built          | ML            | Medium   |
| 24 | src/training/mlflow_tracking_hooks.py            | tracking URL and run-tag helpers               | training and eval                       | MLflow behavior fragments               | ML            | Medium   |
| 25 | src/monitoring/middleware.py                     | middleware and Prometheus metric objects       | every request and metrics scrape        | operational visibility drops            | Solution      | High     |
| 26 | contracts/schemas.py                             | HTTP request and response contracts            | request parsing and docs generation     | API contract breaks                     | Backend       | Critical |
| 27 | contracts/model_interface.py                     | model abstraction and shared dataclasses       | backend-model boundary                  | coupling explodes and tests fail        | Backend + AI  | High     |
| 28 | contracts/errors.py                              | shared model exception types                   | runtime error mapping                   | error handling becomes inconsistent     | Backend + AI  | Medium   |
| 29 | app/sentiment-analysis/main.py                   | main frontend shell                            | every UI session                        | chat UX tool lapses                     | UI/UX         | High     |
| 30 | app/sentiment-analysis/api_client.py             | API client and message state                   | every UI request                        | UIs/cannot call backend                 | UI/UX         | Critical |
| 31 | app/sentiment-analysis/text_and_voice_capture.py | text and voice capture                         | user interaction                        | main input path breaks                  | UI/UX         | High     |
| 32 | app/sentiment-analysis/csv_upload.py             | CSVs upload and result table                   | batch UI flow                           | batch UI disappears                     | UI/UX         | High     |
| 33 | app/sentiment-analysis/types.py                  | typed frontend contract mirror                 | compile/runtime and runtime assumptions | typed drift and bugs                    | UI/UX         | Medium   |
| 34 | app/sentiment-analysis/gateway.py                | gateway had static hosting config              | frontend container runtime              | API routing and uploads fail            | Solution      | High     |
| 35 | docker-compose.yml                               | full stack orchestration                       | integrated deployment                   | no standard local stack                 | Solution      | Critical |
| 36 | Dockerfile                                       | backend runtime image build                    | image build                             | deployable runtime missing              | Solution      | Critical |
| 37 | Dockerfile.train                                 | training image build                           | training container use                  | reproducible training container missing | Solution + ML | High     |
| 38 | dvc.yaml                                         | end-to-end ML pipeline DAG                     | DVC execution                           | stage orchestration disappears          | ML            | Critical |
| 39 | params.yaml                                      | shared config for data, eval, training, MLflow | data and eval stages                    | config defaults fragment                | ML            | High     |
| 40 | requirements.txt                                 | Python dependency set                          | install and CI                          | environment build fails                 | Tech Lead     | High     |
| 41 | .github/workflows/automated_regression_gate      | automated regression gate                      | CI runs                                 | change safety drops                     | Backend       | Medium   |

| #  | File                                                   | Why it exists                           | When it executes            | What breaks if removed                  | Owner    | Risk   |
|----|--------------------------------------------------------|-----------------------------------------|-----------------------------|-----------------------------------------|----------|--------|
| 42 | infra/prometheus/prometheus.yml                        | script target config                    | monitoring runtime          | metrics stop scraping                   | Solution | Medium |
| 43 | infra/prometheus/alertmanager.yml                      | alert thresholds                        | monitoring runtime          | alerting disappears                     | Solution | Medium |
| 44 | infra/grafana/provisioning/datasources/datasources.yml | Grafana source bootstrap                | Grafana startup             | dashboards queries fail                 | Solution | Medium |
| 45 | infra/grafana/provisioning/dashboards/dashboards.json  | operational dashboards                  | Grafana startup and runtime | no default dashboard view               | Solution | Medium |
| 46 | tests/test_api.py                                      | end-to-end API regression tests         | CI and local test runs      | route regressions escape faster         | Backend  | Medium |
| 47 | tests/scripts/test_training_f1_score.py                | training F1 score regression coverage   | CI and local test runs      | training regressions escape faster      | ML       | Medium |
| 48 | tests/data/test_validation.py                          | data quality guard regression tests     | CI and local test runs      | quality-rule drift escapes faster       | ML       | Medium |
| 49 | tests/model/test_throughput.py                         | throughput target evidence              | benchmark test runs         | perf expectations lose executable proof | AI       | Medium |
| 50 | app/sentiment-analysis/frontend/package.json           | frontend dependency and script manifest | frontend install and build  | UI cannot build reproducibly            | UI/UX    | Medium |

## 5.2 Criticality Notes

The highest criticality cluster is `src/main.py`, `src/model/baseline.py`, `contracts/schemas.py`, `docker-compose.yml`, `Dockerfile`, `dvc.yaml`, and `src/scripts/run_finetuning.py`. These files define the runtime contract, the model execution path, and the artifact-production lifecycle.

## 6 Part 6 - Execution Trace

### 6.1 Application Startup

1. Uvicorn imports `src.main:app`.
2. FastAPI registers middleware, routes, and exception handlers.
3. lifespan handler reads environment variable `MODEL_MODE`; default is `onnx`.
4. `ModelConfig(mode=mode)` is created.
5. `BaselineModelInference(config)` is instantiated.
6. `BaselineModelInference._load_model()` loads either ONNX sessions or Hugging Face/PEFT model state.

7. `preload()` eagerly loads the ABSA pipeline.

## 6.2 Health Check

- File: `src/main.py`
- Function: `health_check()`
- Input: HTTP GET
- Output: `HealthResponse`
- Exception path: dependency raises 503 if global model is not ready
- Owner: Backend

## 6.3 Predict

- File: `src/main.py`
- Functions: `predict()`, `resolve_request_language()`
- Core model function: `BaselineModelInference.predict_single()`
- Input: `PredictRequest(text, lang)`
- Output: `PredictResponse`
- Exception path: 400 on unsupported language, 500 on model failure
- Owner: Backend + AI

## 6.4 Explain

- File: `src/main.py`
- Function: `explain()`
- Core model function: `get_shap_explanation()`
- Input: `ExplainRequest`
- Output: token list, SHAP values, base value, latency
- Exception path: model error, tokenizer absence, SHAP runtime error
- Owner: Backend + AI

## 6.5 Batch Predict

- File: `src/main.py`
- Function: `batch_predict()`



- Input: multipart CSV upload
- Output: synchronous `BatchPredictResponse`
- Notes: row cap 500; empty rows converted to failed results; ABSA skipped for performance
- Exception path: invalid CSV, missing `text` column, model batch failure
- Owner: Backend

## 6.6 Evaluation

- Runtime trigger file: `src/main.py`
- Background imports: `src/model/evaluate.py`
- Input: POST no body
- Output: immediate started status and separate in-memory status endpoint
- Gap: persistent job tracking is **Not Implemented**
- Owner: Backend + ML

## 6.7 Training

- File: `src/scripts/run_finetuning.py`
- Main flow: parse args, resolve task config, load raw data, dedup, stratify, oversample, tokenize, build PEFT model, train, evaluate, save adapter, log MLflow
- Inputs: task name, raw CSVs, params, environment
- Outputs: adapter directory, metrics, MLflow run
- Owner: ML

## 6.8 ONNX Export

- CLI file: `src/scripts/export_onnx.py`
- Export implementation: `src/model/onnx_exporter.py`
- Steps: load base model, load adapter, merge, save temp merged model, export FP32 ONNX, quantize to INT8
- Outputs: `models/onnx/*`
- Owner: AI

## 6.9 Sequence Diagrams

```

1 sequenceDiagram
2   participant App as FastAPI app
3   participant Life as lifespan()
4   participant Model as BaselineModelInference
5   participant ONNX as OnnxInferenceSession or HF
6   App->>Life: startup
7   Life->>Model: instantiate with MODEL_MODE
8   Model->>ONNX: load artifacts
9   Life->>Model: preload ABSA
10  Life-->>App: ready

```

```

1 sequenceDiagram
2   participant FE as Frontend
3   participant API as /predict
4   participant LD as LanguageDetector
5   participant INF as BaselineModelInference
6   FE->>API: PredictRequest
7   API->>LD: detect if lang missing
8   API->>INF: predict_single()
9   INF-->>API: PredictionResult
10  API-->>FE: PredictResponse

```

```

1 sequenceDiagram
2   participant CSV as Uploaded CSV
3   participant API as /batch_predict
4   participant PD as pandas
5   participant INF as BaselineModelInference
6   CSV->>API: multipart file
7   API->>PD: read_csv()
8   API->>INF: predict_batch(skip_absa=true)
9   INF-->>API: result list
10  API-->>CSV: BatchPredictResponse

```

```

1 sequenceDiagram
2   participant DVC
3   participant Train as run_finetuning.py
4   participant Eval as evaluate_finetuned.py
5   participant Export as export_onnx.py
6   DVC->>Train: finetune task
7   Train-->>Eval: adapters
8   Eval-->>Export: accepted adapter output

```

## 7 Part 7 - Frontend Handbook

### 7.1 Owner

Nguyen Le Hong Nhi

## 7.2 Architecture

The frontend is an Angular standalone-component application. The key pattern is a thin app shell plus a single service maintaining the message stream.

## 7.3 Components

- AppComponent: layout, health polling, history, overlay control, message rendering
- ChatInputComponent: text input, language selector, browser speech recognition, send events
- BatchUploadComponent: CSV upload, summary cards, results export
- MessageBubbleComponent: message rendering and explainability display

## 7.4 Services and State Flow

- SentimentAnalysisService stores messages in a signal and persists history in local storage.
- API calls go to Nginx-relative paths such as `/api/predict`, `/api/explain`, and `/api/batch_predict`.
- health polling runs from the app shell until the backend returns healthy.

## 7.5 Explainability UI

Explain is not precomputed in the UI. The service finds the preceding user message for a bot reply and then calls `/api/explain`. That means explanation depends on chat history order, not on a stable backend-issued prediction id.

## 7.6 Batch Upload UI

Batch upload is modal-driven. CSV selection is local, upload is synchronous, and the result table is rendered in memory. A durable batch-job UX is **Not Implemented**.

## 7.7 Key Risks

- The language selector emits `lang` but the service drops it on prediction requests.
- The UI offers many languages while backend runtime supports only `en` and `vi`.
- Message history persistence can hide backend readiness by reusing prior chat state.
- Automated frontend coverage is limited.

## 7.8 Extension Points

- wire explicit language end-to-end
- add true batch job polling and downloadable artifacts

- attach explain requests to prediction ids instead of chat adjacency
- split operational settings into authenticated admin surfaces

## 8 Part 8 - Backend Handbook

---

### 8.1 Owner

Pham Duc Long

### 8.2 FastAPI Structure

The backend is a single FastAPI app in `src/main.py`. It owns:

- startup and global model lifecycle
- request validation through Pydantic schemas
- dependency injection for model access
- exception mapping
- monitoring middleware registration

### 8.3 Contracts and Validation

Contracts live in `contracts/schemas.py`. The backend trusts these schemas for OpenAPI and runtime parsing. Health, predict, explain, batch, and evaluation surfaces are all defined through this contract layer.

### 8.4 API Ownership Map

| Endpoint                   | Purpose                                | Owner              |
|----------------------------|----------------------------------------|--------------------|
| GET /health                | readiness and model state              | Backend            |
| POST /predict              | main single-text inference             | Backend + AI       |
| POST /explain              | SHAP explanation path                  | Backend + AI       |
| POST /batch_predict        | synchronous CSV batch scoring          | Backend            |
| GET /batch_status/{job_id} | mocked status response                 | Backend            |
| POST /evaluate             | trigger background baseline evaluation | Backend + ML       |
| GET /evaluate/status       | in-memory evaluation status            | Backend            |
| GET /metrics               | Prometheus metrics scrape              | Backend + Solution |

### 8.5 Error Handling

- `UnsupportedLanguageError` maps to HTTP 400.
- `ModelError` maps to HTTP 500.
- batch parsing failures raise HTTP 400.

- evaluate conflict raises HTTP 409 if a run is already active.

## 8.6 Backend Gaps

- Authentication: **Not Implemented**
- Authorization: **Not Implemented**
- Durable evaluation status store: **Not Implemented**
- Durable async batch processing: **Not Implemented**
- Request rate limiting: **Not Implemented**

## 9 Part 9 - AI Inference Handbook

---

### 9.1 Owner

Do Quoc Trung

### 9.2 Pipeline

Input text follows this path:

Input text → language selection → tokenization → tensors or NumPy arrays → logits → softmax probabilities → label selection → optional ABSA and sarcasm → optional SHAP

### 9.3 Tokenization and Tensors

- HF mode: tokenizer returns PyTorch tensors, truncated at `max_length`
- ONNX mode: tokenizer returns NumPy arrays for `onnxruntime`
- batching uses padding only when predicting lists of texts

### 9.4 ONNX Runtime

- provider selection prefers CUDA, then CoreML for MPS, then CPU
- sentiment and sarcasm ONNX sessions are loaded separately if available
- INT8 path exists through quantized exported models

### 9.5 Prediction Logic

- sentiment label map is index-based: negative, neutral, positive
- sarcasm in finetuned mode uses a dedicated adapter or ONNX session
- sarcasm training uses a 3-logit shared-head design while runtime only consumes the first two logits

## 9.6 ABSA

ABSA is implemented through a zero-shot classification pipeline, not through a dedicated aspect model trained in-repo. Aspect detection and aspect-sentiment inference each require extra zero-shot passes, which is why batch mode disables ABSA.

## 9.7 SHAP

SHAP is built on demand using a prediction callback and tokenizer. There is no caching, no precomputation, and no request budgeting beyond general route execution time.

## 9.8 Language Detection

Language detection is lightweight and heuristic. It uses Vietnamese diacritic detection and token-count minimums. This is sufficient for a fallback but is not enterprise-grade language identification.

## 9.9 Latency Analysis

| Path                                           | Latency implication                                        |
|------------------------------------------------|------------------------------------------------------------|
| Startup                                        | highest cost is model/session load plus ABSA warm-up       |
| Single predict, sentiment only                 | moderate; bounded by tokenizer + model forward             |
| Single predict with ABSA                       | high; multiple zero-shot passes added                      |
| Explain SHAP                                   | very high; explainer construction and repeated model calls |
| Batch predict with <code>skip_absa=true</code> | throughput-oriented and benchmarked in tests               |
| Batch predict with ABSA                        | intentionally avoided in backend implementation            |

# 10 Part 10 - ML Handbook

## 10.1 Owner

Duong Binh An

## 10.2 Dataset Strategy

- SemEval restaurant XML is used for aspect sentiment data
- tweet\_eval irony is used for sarcasm
- multilingual-sentiments English plus UIT-VSFC Vietnamese are used for sentiment finetuning

## 10.3 DVC

`dvc.yaml` is the real training and evaluation backbone. It orchestrates download, preprocess, validate, baseline evaluation, multilingual sentiment and sarcasm download, eval preparation, finetuning, finetuned evaluation, ONNX export, and benchmark stages.

## 10.4 MLflow

MLflow is used in two different ways:

- baseline evaluation logs metrics and artifacts
- finetuning runs log training metadata and parameters

Tracking URI resolution differs between runtime, local CLI, and Docker training contexts.

## 10.5 LoRA and PEFT

Finetuning uses PEFT adapters on top of `xlm-roberta-base`. LoRA config is centralized. This keeps artifact size smaller than full-model copies and supports separate sentiment and sarcasm adapters.

## 10.6 Evaluation and Fairness

Evaluation produces:

- overall macro-F1
- per-language F1
- per-language gap
- confusion matrices
- classification reports

Fairness here means language-slice disparity inspection, not a broader governance framework.

## 10.7 Training Lifecycle

```
1 flowchart LR
2   Raw --> Normalize
3   Normalize --> Validate
4   Validate --> Split
5   Split --> Balance
6   Balance --> Tokenize
7   Tokenize --> Train
8   Train --> Evaluate
9   Evaluate --> SaveAdapter
10  SaveAdapter --> ExportONNX
```

## 10.8 ML Gaps

- Model registry: **Not Implemented**
- Formal promotion approval workflow: **Not Implemented**
- Signed artifacts or checksum verification: **Not Implemented**

## 11 Part 11 - Solution Engineering Handbook

---

### 11.1 Owner

Duong Hong Quan

### 11.2 End-to-End Integration

Solution engineering owns the integration story more than any single code module: frontend routing, backend availability, observability, Compose topology, and environment coherence.

### 11.3 Deployment Architecture

- frontend container serves Angular through Nginx
- backend container runs FastAPI with model artifacts and processed data
- Prometheus scrapes the backend
- Grafana reads Prometheus
- MLflow runs as a separate service
- training container is profile-gated and separate from inference runtime

### 11.4 Monitoring Strategy

Current monitoring covers:

- request count by method, endpoint, status
- request latency
- model inference latency by language label
- simple alerts for high 5xx rate and high inference latency

Tracing, structured request logs, and SLO dashboards are **Not Implemented**.

### 11.5 Operational Readiness Assessment

The stack is good for local integration, internal demo, and controlled pilot. It is not ready for enterprise-grade multi-environment operations because secret handling, auth, durable job control, and deployment rollout strategy are incomplete.

## 12 Part 12 - Ownership Matrix

---



## 12.1 Folder Ownership Matrix

| Scope                        | Primary folders                               | Owner         |
|------------------------------|-----------------------------------------------|---------------|
| UI surface                   | app/                                          | UI/UX Lead    |
| Backend contract and routes  | src/main.py, contracts/                       | Backend Lead  |
| Inference runtime            | src/model/, models/                           | AI Lead       |
| Data and training            | src/data/, src/training/, src/scripts/, data/ | ML Lead       |
| Deployment and observability | infra/, docker-compose.yml, Dockerfiles       | Solution Lead |
| Cross-cutting quality        | tests/, docs/                                 | Tech Lead     |

## 12.2 Service Ownership Matrix

| Service                    | Responsibility                       | Owner            |
|----------------------------|--------------------------------------|------------------|
| Angular + Nginx frontend   | user interaction and gateway routing | UI/UX + Solution |
| FastAPI runtime            | API availability and contract        | Backend          |
| Model runtime in-process   | inference correctness and latency    | AI               |
| Training profile container | adapter production                   | ML               |
| Prometheus / Grafana       | monitoring and alerts                | Solution         |
| MLflow                     | experiment visibility                | ML               |

## 12.3 Operational, Security, Documentation, and Knowledge Ownership

- operational ownership: Solution Lead primary, Backend and AI secondary
- security ownership: currently shared; explicit security owner is **Not Implemented**
- documentation ownership: Tech Lead primary, area leads secondary
- knowledge ownership: AI and ML currently hold the deepest artifact knowledge; bus factor is low

## 13 Part 13 - RACI Matrix

| Workstream           | Tech | ML  | AI | Solution | Backend | UI/UX |
|----------------------|------|-----|----|----------|---------|-------|
| Feature development  | A    | C   | C  | C        | R       | R     |
| Model training       | C    | A/R | C  | I        | I       | I     |
| Model deployment     | A    | C   | R  | R        | C       | I     |
| API changes          | A    | I   | C  | C        | R       | C     |
| Production release   | A    | C   | C  | R        | R       | C     |
| Monitoring           | C    | I   | C  | A/R      | R       | I     |
| Security             | A    | I   | C  | R        | R       | C     |
| Incident response    | A    | C   | R  | R        | R       | I     |
| Architecture changes | A/R  | C   | C  | C        | C       | C     |

## 14 Part 14 - Data and Model Lineage

```

1 flowchart LR
2   Raw[Raw Data]
3   Validation1[Raw Schema Validation]
4   Preprocess[Preprocessing]
5   Validation2[Processed Quality Validation]
6   Training[Finetuning]
7   Evaluation[Evaluation]
8   MLflow[MLflow]
9   Export[Export]
10  ONNX[ONNX Artifacts]
11  Runtime[Runtime Load]
12  Prediction[Prediction]
13  Raw --> Validation1 --> Preprocess --> Validation2 --> Training -->
    Evaluation --> MLflow --> Export --> ONNX --> Runtime --> Prediction

```

| Stage         | Producer                           | Consumer                    | Owner        | Artifact / Lifecycle             |
|---------------|------------------------------------|-----------------------------|--------------|----------------------------------|
| Raw data      | src/data/downloader.py             | preprocessing and training  | ML           | CSV files in data/raw/           |
| Validation    | raw schema and processed validator | training and review         | ML           | reports under data/reports/      |
| Preprocessing | PreprocessingPipeline              | baseline eval and training  | ML           | data/processed/                  |
| Training      | run_finetuning.py                  | evaluation and export       | ML           | adapters in models/adapters/     |
| Evaluation    | evaluate.py, evaluate_finetuned.py | MLflow and reports          | ML + AI      | metrics JSON and artifacts       |
| MLflow        | baseline and training flows        | engineers and reviewers     | ML           | local or service-tracked runs    |
| Export        | onnx_exporter.py                   | runtime container           | AI           | FP32 and INT8 ONNX dirs          |
| Runtime load  | BaselineModelInference             | predict_load_explore, batch | AI           | in-memory session or model state |
| Prediction    | predict_single(), predict_batch()  | frontend and callers        | AI + Backend | response payloads                |

## 15 Part 15 - Security Review

| Finding                                      | Severity | Owner              | Mitigation                                                             |
|----------------------------------------------|----------|--------------------|------------------------------------------------------------------------|
| Authentication on API surface                | Critical | Backend + Solution | Not Implemented; add authenticated gateway and token enforcement       |
| Authorization for admin-like evaluation flow | High     | Backend            | Not Implemented; restrict /evaluate and status to authorized operators |
| Permissive CORS wildcard                     | High     | Backend            | replace with environment allowlist                                     |

| Finding                                                | Severity | Owner              | Mitigation                                                              |
|--------------------------------------------------------|----------|--------------------|-------------------------------------------------------------------------|
| Default Grafana admin password                         | High     | Solution           | move to secrets management and rotate at bootstrap                      |
| Build-time DVC/DagsHub credentials                     | High     | Solution           | stop passing long-lived secrets as build args                           |
| Supply chain exposure through broad dependency install | Medium   | Tech Lead          | pin and scan dependencies regularly                                     |
| CSV upload content trust                               | Medium   | Backend            | add file-size, MIME, and row-policy validation at gateway and app layer |
| Model artifact integrity verification                  | Medium   | AI + ML            | add checksum and promotion gates                                        |
| Container hardening beyond default slim images         | Medium   | Solution           | run as non-root, tighten permissions, add image scanning                |
| Rate limiting and abuse control                        | High     | Backend + Solution | Not Implemented; enforce at gateway and app                             |

## 16 Part 16 - Performance Review

### 16.1 Startup Time

Exact startup measurements are **Not Implemented** as committed telemetry. Source analysis shows the dominant costs are model/session loading and ABSA pipeline preload.

### 16.2 Memory Usage

Exact memory profiling is **Not Implemented**. Likely dominant memory holders are model weights, tokenizer assets, ONNX sessions, and zero-shot model state.

### 16.3 CPU Usage

The runtime is CPU-oriented by default in Dockerfiles. Compose explicitly allocates high CPU limits to the backend. SHAP and ABSA are the most CPU-expensive paths.

### 16.4 Inference Latency

Source and tests indicate:

- sentiment-only batch throughput target is at least 100 samples/sec in benchmark tests
- full pipeline throughput with ABSA is intentionally much slower
- Prometheus alerting considers average inference latency above 500 ms a warning

### 16.5 Batch Throughput

Batch predict is synchronous and uses `asyncio.to_thread()` for backend responsiveness. It still performs CPU-bound inference in-process. There is no distributed worker model.

## 16.6 SHAP Cost

SHAP is high cost because it builds an explainer on demand and performs repeated model evaluations. This path is suitable for explain-on-demand, not for high-throughput production.

## 16.7 ABSA Cost

ABSA uses zero-shot classification for both aspect detection and sentiment-per-aspect. That makes it accurate enough for a demo architecture but expensive for scale.

## 16.8 Scalability Limits

- single backend process owns model state
- no horizontal job partitioning
- no queue-backed batch workers
- no caching for SHAP or aspect extraction

## 17 Part 17 - Operations Handbook

| Incident              | Symptoms                              | Root Cause                                        | Debug Steps                                                         | Resolution                                       | Owner         |
|-----------------------|---------------------------------------|---------------------------------------------------|---------------------------------------------------------------------|--------------------------------------------------|---------------|
| API failure           | health fails, UI overlay never clears | model load failure, crash, bad image              | inspect backend logs, call /health, verify artifacts and MODEL_MODE | restore valid image or artifacts                 | Backend       |
| Model failure         | 500 on predict or explain             | corrupted tokenizer, adapter, or ONNX file        | reproduce request, inspect stack, test local model init             | redploy last known-good artifact                 | AI            |
| Latency spike         | alert fires, user response slows      | ABSA/SHAP cost, CPU saturation, large batch       | compare route latency, inspect active route mix, check host CPU     | temporarily disable heavy paths or scale runtime | AI + Solution |
| Monitoring failure    | empty dash-board or no alerts         | scrape config drift, metrics endpoint failure     | hit /metrics, check Prometheus target status, inspect datasource    | restore scrape target or dash-board config       | Solution      |
| Deployment failure    | one or more services unhealthy        | config drift, secret issue, missing data mount    | inspect Compose logs and service order                              | revert deployment input or image                 | Solution      |
| Batch failure         | CSV upload returns 400 or 500         | malformed CSV or missing text column              | validate sample input, inspect parser exception, check Nginx limit  | repair input validation or backend parser        | Backend       |
| Data pipeline failure | DVC stage stops                       | missing raw files, validation failure, path drift | run stage locally, inspect report JSON and file existence           | correct data or configuration                    | ML            |

## 18 Part 18 - Technical Debt Register

| Level    | Description                              | Evidence                                           | Impact                      | Owner     | Recommended Fix                        | Priority |
|----------|------------------------------------------|----------------------------------------------------|-----------------------------|-----------|----------------------------------------|----------|
| Critical | Public unauthenticated runtime           | no auth code in backend or gateway                 | enterprise blocker          | Backend   | add authn/authz                        | P0       |
| High     | Mocked batch status contract             | /batch_status returns hard-coded completed payload | misleading async semantics  | Backend   | implement queue or remove endpoint     | P1       |
| High     | Frontend language selection gap          | event carries lang; service drops it               | user expectation mismatch   | UI/UX     | pass lang end-to-end                   | P1       |
| High     | Default Grafana password                 | Compose sets GF_SECURITY_ADMIN_PASSWORD=admin      | security risk               | Solution  | move to secret store                   | P1       |
| High     | Public evaluation trigger                | /evaluate accessible without guard                 | abuse and data leakage risk | Backend   | restrict to operator role              | P1       |
| Medium   | In-memory evaluation status              | state resets on process restart                    | operator confusion          | Backend   | use durable status store               | P2       |
| Medium   | No tracing                               | only Prometheus counters and histograms exist      | reduced incident visibility | Solution  | add tracing and correlation ids        | P2       |
| Medium   | Artifact integrity is implicit           | no checksum or signature verification              | bad model promotion risk    | AI + ML   | add artifact validation                | P2       |
| Medium   | Frontend explain is chat-order dependent | previous user message lookup in service            | fragile UX semantics        | UI/UX     | use prediction id mapping              | P2       |
| Low      | Legacy docs still overstate behavior     | README and contract drift from implementation      | onboarding friction         | Tech Lead | clean doc set and deprecate stale docs | P3       |

## 19 Part 19 - Organizational Review

### 19.1 Team Structure

The team structure is reasonable for a compact ML product: one lead per major discipline. The weakness is not the structure itself; it is the concentration of low-level system knowledge in a few files and a few owners.

### 19.2 Knowledge Distribution and Bus Factor

- bus factor for inference internals is low

- bus factor for training and artifact promotion is low
- bus factor for deployment and observability is moderate
- UI ownership is clearer but still coupled to backend response details

### 19.3 Ownership Gaps

- dedicated security owner: **Not Implemented**
- formal release manager role: **Not Implemented**
- model promotion board or approval workflow: **Not Implemented**

### 19.4 Recommendations

- add CODEOWNERS by folder
- require backup reviewer across AI/ML and Backend/Solution boundaries
- create a formal artifact-promotion checklist
- make security accountability explicit

## 20 Part 20 - Production Readiness Scorecard

| Area                        | Score     | Weight      | Reason                                                                           |
|-----------------------------|-----------|-------------|----------------------------------------------------------------------------------|
| Architecture                | 74        | 0.12        | coherent layering and artifact separation, but single-process constraints remain |
| Security                    | 30        | 0.15        | major blocking gaps remain                                                       |
| Reliability                 | 58        | 0.10        | health exists, durability does not                                               |
| Performance                 | 63        | 0.10        | ONNX and batching help, SHAP and ABSA are expensive                              |
| Testing                     | 76        | 0.10        | good Python coverage, thin frontend coverage                                     |
| Monitoring                  | 71        | 0.08        | Prometheus and Grafana exist, tracing absent                                     |
| ML Lifecycle                | 79        | 0.10        | DVC, MLflow, training, eval, export are real                                     |
| Deployment                  | 60        | 0.08        | Compose is functional but not enterprise-grade                                   |
| Operations                  | 56        | 0.08        | basic runbooks are possible, automation is limited                               |
| Documentation               | 72        | 0.04        | improved by this handbook, legacy drift still exists                             |
| Ownership                   | 69        | 0.05        | lead mapping is clear, knowledge concentration remains                           |
| <b>Weighted Final Score</b> | <b>63</b> | <b>1.00</b> | advanced MVP, not enterprise production ready                                    |

## 21 Part 21 - Roadmap

### 21.1 3-Month Roadmap

- Authentication, authorization, and rate limiting. Owner: Backend + Solution. Priority: P0. Business value: unblock secure pilot.

- Replace mocked batch status with durable batch architecture. Owner: Backend. Priority: P1. Risk reduction: removes false operational promise.
- Fix language propagation and supported-language UX. Owner: UI/UX + Backend + AI. Priority: P1. Business value: trust and correctness.

## 21.2 6-Month Roadmap

- Add artifact registry, integrity checks, and promotion workflow. Owner: AI + ML. Priority: P1.
- Add tracing and operational correlation identifiers. Owner: Solution + Backend. Priority: P2.
- Isolate admin evaluation functions from public runtime. Owner: Backend + Solution. Priority: P1.

## 21.3 12-Month Roadmap

- Move beyond Compose to managed deployment with stronger secret handling and rollout control. Owner: Solution. Priority: P1.
- Add scalable worker pattern for batch and explainability-heavy jobs. Owner: Backend + AI. Priority: P2.
- Formalize multilingual roadmap only after data and evaluation parity is demonstrable. Owner: ML + AI + UI/UX. Priority: P2.

## 22 Part 22 - Onboarding Handbook

| Role and Milestone       | Learning Plan                                                                                                 |
|--------------------------|---------------------------------------------------------------------------------------------------------------|
| Tech Lead Day 1          | Read <code>src/main.py</code> , <code>docker-compose.yml</code> , <code>dvc.yaml</code> , this handbook.      |
| Tech Lead Day 30         | Be able to arbitrate architecture, security, and release decisions.                                           |
| ML Engineer Day 1        | Read <code>src/data/</code> , <code>src/training/</code> , <code>params.yaml</code> , <code>dvc.yaml</code> . |
| ML Engineer Day 30       | Run download, preprocess, validate, finetune, evaluate, and explain lineage.                                  |
| AI Engineer Day 1        | Read <code>src/model/</code> and benchmark-related tests.                                                     |
| AI Engineer Day 30       | Diagnose model load, ONNX, SHAP, ABSA, and latency regressions independently.                                 |
| Backend Engineer Day 1   | Read <code>src/main.py</code> , <code>contracts/</code> , <code>tests/test_api.py</code> .                    |
| Backend Engineer Day 30  | Own route changes, validation, metrics, and failure handling.                                                 |
| Solution Engineer Day 1  | Read Dockerfiles, Compose, Nginx config, Prometheus and Grafana provisioning.                                 |
| Solution Engineer Day 30 | Operate the full local stack and lead deployment debugging.                                                   |
| UI/UX Engineer Day 1     | Read app shell, service, input, batch, and model contracts.                                                   |
| UI/UX Engineer Day 30    | Own chat flow, batch UX, explain UX, and contract alignment.                                                  |

## 23 Part 23 - CTO Review

### 23.1 Would I approve production launch?

No, not for open production or enterprise production. I would approve internal demonstration and a tightly controlled pilot after security hardening.

### 23.2 Would I approve enterprise customers?

No. Security and operational durability are not sufficient yet.

### 23.3 Would I approve scaling?

Not yet. The architecture is directionally scalable, but the current operating model is not.

### 23.4 What concerns me most?

The main concern is appearance-versus-reality. The repository looks mature enough to create false confidence, but critical controls remain missing or partially implemented.

### 23.5 What would be my first investment?

My first investment would be platform hardening: authentication, rate limiting, admin-surface isolation, and artifact-governance improvements. That creates the foundation for every later business decision.

## Implementation Discrepancies

---

- README health example lists only [ "en" ]; implementation supports `en` and `vi`.
- README describes batch processing like an async job; implementation is synchronous request-response and the status endpoint is mocked.
- frontend exposes many language options; backend supports only `en` and `vi`; frontend prediction requests usually omit explicit `lang`.
- Compose uses internal MLflow URL `http://mlflow:5000`, host publishes 5005, and params default to `http://localhost:5000`. Local and container assumptions are not unified.
- `contracts/README.md` shows `/api/v1/*` examples; implementation uses root paths such as `/predict` and `/health`, with Nginx adding the frontend-facing `/api/` prefix.